

Algoritmi e Strutture Dati – 20/07/2026

Esercizio A1 – Massima distanza – Punti ≥ 8

Soluzione banale Una soluzione banale (spesso ne esiste una) consiste nel considerare tutte le coppie di indici (i, j) con $0 \leq i < j \leq n - 1$, e aggiornare la risposta ogni volta che $A[i] = A[j]$. Il numero di coppie è $\Theta(n^2)$, quindi il tempo richiesto è $\mathcal{O}(n^2)$.

```
int maxDistance(int[] A, int n)
precondition: n ≥ 0
int maxSoFar = 0
for i = 0 to n - 2 do
    for j = i + 1 to n - 1 do
        if A[i] == A[j] then
            maxSoFar = max(maxSoFar, j - i)
return maxSoFar
```

Questa versione, banalissima e risolvibile senza problemi dopo un corso di Programmazione 1, prende il 70%.

Soluzione efficiente Una soluzione più efficiente usa una tabella hash per memorizzare, per ciascun valore x , la posizione della sua *prima* occorrenza $first[x]$. Scorrendo il vettore da sinistra a destra, quando si incontra $x = A[i]$:

- se x non è mai stato visto, si salva $first[x] = i$;
- altrimenti, la distanza massima ottenibile con l'occorrenza corrente è $i - first[x]$, e si può aggiornare $maxSoFar$.

Ogni operazione sulla tabella hash richiede tempo atteso $\mathcal{O}(1)$, quindi il tempo totale è $\mathcal{O}(n)$, con spazio aggiuntivo $\mathcal{O}(k)$ dove k è il numero di valori distinti in A .

```
int maxDistance(int[] A, int n)
precondition: n ≥ 0
int maxSoFar = 0
HASH first = Hash()
for i = 0 to n - 1 do
    if first.lookup(A[i]) == nil then
        first.insert(A[i], i)
    else
        int distance = i - first.lookup(A[i])
        maxSoFar = max(maxSoFar, distance)
return maxSoFar
```

Questa versione prende il 100%.

Esercizio A2 – Complessità – Punti ≥ 10

Il Master Theorem nella sua versione base non è applicabile, perché il termine non ricorsivo $f(n) = \log n$ non è della forma $\Theta(n^d)$ per alcuna costante d .

Anche la versione estesa non è direttamente applicabile. Infatti,

$$n^{\log_b a} = n^{\log_2 1} = 1,$$

e $f(n) = \log n$ è asintoticamente maggiore di 1, ma non è polinomialmente maggiore: non esiste infatti alcuna costante $\varepsilon > 0$ tale che $\log n = \Omega(n^\varepsilon)$. D'altra parte, $\log n$ non è nemmeno $\Theta(1)$, quindi non ricadiamo nel caso bilanciato.

Fare correttamente un'affermazione del genere senza ulteriori specifiche è valutato 60%. Affermare invece che è applicabile dà origine a un voto fra 0% e 40%, a seconda di come è scritto. Avventurarsi in ulteriori affermazioni discutibili dà origine a un voto fra 0% e 50%.

Dimostrazione del limite inferiore È facile vedere che l'equazione di ricorrenza ha una componente non ricorsiva $\log n$, quindi è possibile affermare che $T(n) = \Omega(\log n)$. In caso vi fermiate qui, la valutazione è 60%.

Dimostrazione del limite superiore Un limite superiore, anche se non stretto, può essere dimostrato con il metodo di sostituzione. Tentiamo di dimostrare che

$$T(n) = \mathcal{O}(n),$$

ovvero che esiste una costante $c > 0$ tale che

$$T(n) \leq cn.$$

Ipotesi induttiva. Supponiamo che

$$T(k) \leq ck$$

Passo induttivo. Utilizzando l'ipotesi induttiva, otteniamo

$$\begin{aligned} T(n) &= T(n/2) + \log n \\ &\leq c \frac{n}{2} + \log n && \text{Sostituzione} \\ &\leq c \frac{n}{2} + n && \text{Poiché } \log n \leq n \\ &\stackrel{?}{\leq} cn && \text{Obiettivo} \\ &\iff \frac{c}{2} + 1 \leq c \\ &\iff c \geq 2. \end{aligned}$$

Caso base. Per $n = 1$ si ha

$$T(1) = 1 \leq c,$$

che è vero per $c \geq 1$.

Scegliendo quindi $c = 2$, entrambe le condizioni sono soddisfatte e otteniamo

$$T(n) \leq 2n.$$

Pertanto

$$T(n) = \mathcal{O}(n).$$

Se dimostrate correttamente che $T(n) = \mathcal{O}(n)$, la valutazione sale al 90%. Ho anche visto $\mathcal{O}(n \log n)$ che è stato valutato 80%.

Analisi per livelli L'analisi per livelli ci dà un limite stretto. La ricorrenza genera una sola chiamata ricorsiva a ogni livello:

Livello	Dimensione	Costo chiamata	N. chiamate	Costo livello
0	n	$\log n$	1	$\log n$
1	$n/2$	$\log(n/2)$	1	$\log(n/2)$
2	$n/4$	$\log(n/4)$	1	$\log(n/4)$
...	...	...	...	...
i	$n/2^i$	$\log(n/2^i)$	1	$\log(n/2^i)$
...	...	...	...	...
$\ell - 1$	2	$\log 2 = 1$	1	1
$\ell = \log n$	1	$T(1)$	1	1

La tabella permette già di ottenere una prima stima. I livelli non foglia sono $\log n$ e ciascuno ha costo almeno 1, quindi

$$T(n) = \Omega(\log n).$$

D'altra parte, ciascun livello costa al più $\log n$; essendoci $\log n$ livelli non foglia, otteniamo

$$T(n) = \mathcal{O}((\log n)^2).$$

Questa analisi circonda quindi la complessità tra $\Omega(\log n)$ e $\mathcal{O}((\log n)^2)$.

Per ottenere un limite stretto, sommiamo i costi dei livelli. Poiché

$$\log\left(\frac{n}{2^i}\right) = \log n - \log 2^i = \log n - i,$$

si ha

$$\begin{aligned}
 T(n) &= T(1) + \sum_{i=0}^{\log n - 1} \log\left(\frac{n}{2^i}\right) \\
 &= 1 + \sum_{i=0}^{\log n - 1} (\log n - i) && \text{Proprietà logaritmo} \\
 &= 1 + \left(\sum_{i=0}^{\log n - 1} \log n \right) - \left(\sum_{i=0}^{\log n - 1} i \right) && \text{Separazione sommatoria} \\
 &= 1 + (\log n)^2 - \frac{(\log n - 1) \log n}{2} && \text{Formula di Gauss} \\
 &= 1 + \frac{(\log n + 1) \log n}{2} && \text{Passaggi algebrici}
 \end{aligned}$$

Pertanto

$$T(n) = \Theta((\log n)^2).$$

In questo caso, ovviamente, la valutazione è 100%.

Esercizio A3 – Parenti serpenti – Punti ≥ 12

L'albero deve essere visitato sia verso il basso, attraverso i figli, sia verso l'alto, attraverso il padre. Poiché in un albero esiste un unico cammino semplice fra ogni coppia di nodi, per evitare di visitare più volte uno stesso nodo è sufficiente ricordare il nodo precedente. Non è quindi necessario mantenere un insieme dei nodi già visitati.

La funzione ausiliaria `crRec()` riceve un nodo t , il nodo precedente p e la distanza residua d . Essa conta il nodo t e tutti i nodi raggiungibili da t mediante al più d archi, senza attraversare l'arco che conduce nuovamente a p .

int countRelatives(**TREE** t , **int** d)

precondition: $t \neq \text{nil}$ and $d \geq 0$ **return** crRec(t , **nil**, d) - 1;

int crRec(**TREE** t , **TREE** p , **int** d)

precondition: $t \neq \text{nil}$ and $d \geq 0$ **int** $count = 1$ % Conta il nodo t **if** $d > 0$ **then** **TREE** $u = t.\text{parent}()$ **if** $u \neq \text{nil}$ and $u \neq p$ **then** $count += \text{crRec}(u, t, d - 1)$ $u = t.\text{leftmostChild}()$ **while** $u \neq \text{nil}$ **do** **if** $u \neq p$ **then** $count += \text{crRec}(u, t, d - 1)$ $u = u.\text{rightSibling}()$ **return** $count$

La funzione visita l'albero come un grafo non orientato, considerando per ogni nodo il padre e tutti i figli. Il parametro p impedisce di tornare immediatamente al nodo precedente e quindi di visitare due volte gli stessi nodi.

A ogni chiamata con distanza residua d , vengono contati esattamente il nodo corrente e tutti i nodi raggiungibili con al più d archi senza attraversare p . Poiché in un albero esiste un unico cammino semplice tra due nodi, ogni nodo viene raggiunto una sola volta. La chiamata iniziale considera tutte le direzioni e la sottrazione finale esclude correttamente il nodo t .

Nel caso peggior, l'algoritmo visita ogni nodo dell'albero una e una sola volta, quindi la complessità è $\mathcal{O}(n)$.

Esercizio B1 – Banco alimentare – Punti ≥ 10

Costruiamo una rete di flusso $G' = (V', E', s, t, c)$ composta dai seguenti nodi:

- una sorgente s ;
- un nodo per ogni magazzino $m \in M$;
- un nodo per ogni centro di accoglienza $c \in C$;
- un pozzo t .

Aggiungiamo i seguenti archi:

- un arco (s, m) di capacità s_m , per ogni magazzino $m \in M$;
- un arco (m, c) di capacità $\min(s_m, d_c)$, per ogni coppia $m \in M, c \in A_m$ (alternativamente, i valori $s_m, d_c, +\infty$ andrebbero bene lo stesso);
- un arco (c, t) di capacità d_c , per ogni centro $c \in C$.

Calcoliamo un flusso massimo f da s a t . Il valore del flusso massimo rappresenta il massimo numero complessivo di pacchi che possono essere distribuiti. Per ogni coppia (m, c) con $c \in A_m$, il numero di pacchi inviati dal magazzino m al centro c è dato da

$$f(m, c).$$

Poiché tutte le capacità sono intere, esiste un flusso massimo intero e quindi ogni valore $f(m, c)$ rappresenta un numero intero di pacchi.

Completezza. Consideriamo una qualunque distribuzione valida e indichiamo con $x_{m,c}$ il numero di pacchi inviati dal magazzino m al centro c . Definiamo un flusso ponendo

$$f(s, m) = \sum_{c \in A_m} x_{m,c}, \quad f(m, c) = x_{m,c}, \quad f(c, t) = \sum_{m: c \in A_m} x_{m,c}.$$

I vincoli sulle scorte garantiscono che $f(s, m) \leq s_m$, mentre i vincoli sui centri garantiscono che $f(c, t) \leq d_c$. La conservazione del flusso vale nei nodi corrispondenti ai magazzini e ai centri. Ogni distribuzione valida di k pacchi induce quindi un flusso ammissibile di valore k . Pertanto, la rete di flusso rappresenta tutte le possibili distribuzioni valide.

Complessità. La rete contiene

$$|V| = n_m + n_c + 2$$

nodi. Nel caso peggiore, ogni magazzino può rifornire tutti i centri, quindi

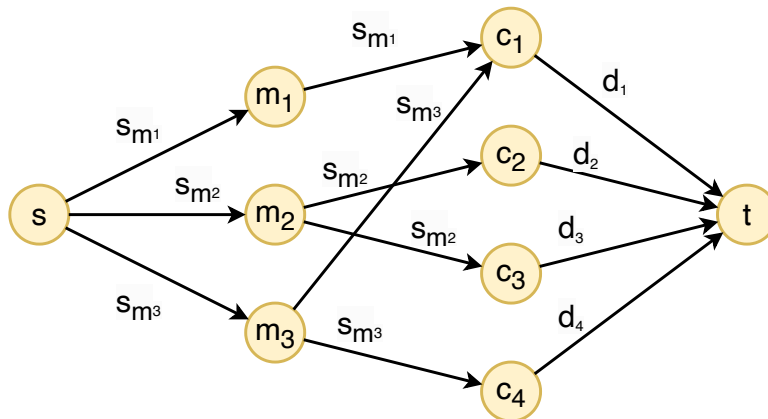
$$|E| \leq n_m + n_c + n_m n_c = \mathcal{O}(n_m n_c).$$

Poiché tutte le capacità sono intere, possiamo utilizzare l'algoritmo di Ford–Fulkerson. Indicando con $|F^*|$ il valore del flusso massimo, la sua complessità temporale è

$$\mathcal{O}(|E|F) = \mathcal{O}(n_m n_c |F^*|),$$

dove

$$|F^*| \leq \min \left\{ \sum_{m \in M} s_m, \sum_{c \in C} d_c \right\}.$$



Esercizio B2 – Acqua alta – Punti ≥ 10

Soluzione semplice: scansione per livelli

Sia $k = \max_{0 \leq i < n} h[i]$ l'altezza massima. Possiamo scandire separatamente i k livelli orizzontali. Per ogni livello ℓ , individuiamo la prima e l'ultima colonna che raggiungono almeno tale livello. Ogni posizione compresa fra queste due colonne e avente altezza minore di ℓ contiene un'unità d'acqua a quel livello.

```
int howMuchWater(int[] h, int n)
```

```
  precondition:  $n \geq 0$  and  $h[i] \geq 0$  per  $0 \leq i < n$ 
```

```
  if  $n < 3$  then
```

```
    return 0
```

```
  int  $k = \max(h)$ 
```

```
  int water = 0
```

```
  for  $\ell = 1$  to  $k$  do
```

```
    int  $L = 0$ 
```

```
    % Prima colonna partendo da sinistra (0) che ha valore almeno  $\ell$ 
```

```
    while  $h[L] < \ell$  do
```

```
       $L = L + 1$ 
```

```
    int  $R = n - 1$ 
```

```
    % Prima colonna partendo da destra ( $n - 1$ ) che ha valore almeno  $\ell$ 
```

```
    while  $h[R] < \ell$  do
```

```
       $R = R - 1$ 
```

```
    for  $i = L + 1$  to  $R - 1$  do
```

```
      if  $h[i] < \ell$  then
```

```
        water = water + 1
```

```
  return water
```

Per ogni livello ℓ , l'algoritmo conta una posizione se e solo se essa è vuota a quel livello ed è delimitata a sinistra e a destra da due colonne di altezza almeno ℓ . Viene quindi contato esattamente ogni quadrato unitario occupato dall'acqua.

Per ogni livello vengono effettuate scansioni che richiedono complessivamente $\mathcal{O}(n)$ tempo. La complessità totale è quindi $\mathcal{O}(kn)$. Poiché k dipende dal valore numerico delle altezze, la complessità è pseudo-polinomiale.

Una versione alternativa, con la stessa complessità ma più semplice, è la seguente:

```
int howMuchWater(int[] h, int n)
```

```
  precondition:  $n \geq 0$  and  $h[i] \geq 0$  per  $0 \leq i < n$ 
```

```
  if  $n < 3$  then
```

```
    return 0
```

```
  int  $k = \max(h)$ 
```

```
  int water = 0
```

```
  for  $\ell = 1$  to  $k$  do
```

```
    boolean found = false
```

```
    int gap = 0
```

```
    for  $i = 0$  to  $n - 1$  do
```

```
      if  $h[i] < \ell$  then
```

```
        if found then
```

```
          gap = gap + 1
```

```
          % A sinistra c'è un muro, spazio per acqua, incrementiamo gap
```

```
        else
```

```
          if found then
```

```
            water = water + gap
```

```
            % Muro a destra per il gap precedente
```

```
            gap = 0
```

```
            % Inizia nuovo gap
```

```
          else
```

```
            found = true
```

```
            % Primo muro
```

```
  return water
```

Questa versione scorre per livelli e poi da sinistra a destra; quando trova il primo muro, inizia a contare acqua "potenziale" in *gap*, che poi aggiungerà a *water* se trova un secondo muro, che chiude il gap precedente e ne inizia uno nuovo.

Soluzione intermedia: scansione per colonne

La soluzione precedente ragiona livello per livello. Possiamo invece considerare una colonna alla volta.

Per ogni posizione i cerchiamo la colonna più alta alla sua sinistra e quella più alta alla sua destra. Siano quindi

$$\begin{aligned} \max_L &= \max_{0 \leq j < i} h[j] \\ \max_R &= \max_{i < j < n} h[j] \end{aligned}$$

L'acqua può raggiungere al più l'altezza $\min(\max_L, \max_R)$. Se tale valore è superiore all'altezza della colonna corrente, la quantità d'acqua trattenuta sopra $h[i]$ è

$$\min(\max_L, \max_R) - h[i].$$

```
int howMuchWater(int[] h, int n)
```

precondition: $n \geq 0$ and $h[i] \geq 0$ per $0 \leq i < n$

```
int water = 0
```

```
for i = 1 to n - 2 do
```

```
    int max_L = max(h, 0, i - 1)
```

```
    int max_R = max(h, i + 1, n - 1)
```

```
    water += max(0, min(max_L, max_R) - h[i])
```

```
return water
```

Per ogni colonna vengono individuate le due barriere più alte che la delimitano. Il livello dell'acqua è determinato dalla più bassa delle due; se tale livello supera l'altezza della colonna, la differenza rappresenta la quantità d'acqua trattenuta sopra di essa.

Per ciascuna delle n colonne vengono effettuate due ricerche del massimo nei sottovettori di destra e sinistra, con costo $\mathcal{O}(n)$. La complessità complessiva è quindi $\Theta(n^2)$.

Soluzione efficiente: programmazione dinamica

La soluzione precedente ricalcola ripetutamente, per ogni colonna, la massima altezza presente alla sua sinistra e alla sua destra. Possiamo evitare queste scansioni memorizzando tali informazioni in due tabelle. Definiamo quindi:

$$\begin{aligned} DP_{left}[i] &= \max_{0 \leq j \leq i} h[j] \\ DP_{right}[i] &= \max_{i \leq j < n} h[j] \end{aligned}$$

Le due tabelle possono essere calcolate mediante le ricorrenze

$$DP_{left}[i] = \begin{cases} h[0] & i = 0 \\ \max(DP_{left}[i-1], h[i]) & i > 0 \end{cases}$$

e

$$DP_{right}[i] = \begin{cases} h[n-1] & i = n-1 \\ \max(DP_{right}[i+1], h[i]) & i < n-1 \end{cases}$$

L'altezza raggiunta dall'acqua nella posizione i è $\min(DP_{left}[i], DP_{right}[i])$. La quantità d'acqua contenuta sopra la colonna è pertanto $\min(DP_{left}[i], DP_{right}[i]) - h[i]$

int howMuchWater(**int**[] *h*, **int** *n*)

precondition: $n \geq 0$ and $h[i] \geq 0$ per $0 \leq i < n$ **if** $n < 3$ **then**└ **return** 0;**int**[] $DP_{left} = \text{new int}[0 \dots n - 1]$ **int**[] $DP_{right} = \text{new int}[0 \dots n - 1]$ $DP_{left}[0] = h[0]$ **for** $i = 1$ **to** $n - 1$ **do**└ $DP_{left}[i] = \max(DP_{left}[i - 1], h[i])$ $DP_{right}[n - 1] = h[n - 1]$ **for** $i = n - 2$ **downto** 0 **do**└ $DP_{right}[i] = \max(DP_{right}[i + 1], h[i])$ **int** *water* = 0**for** $i = 1$ **to** $n - 2$ **do**└ $water += \min(DP_{left}[i], DP_{right}[i]) - h[i]$ **return** *water*

Le ricorrenze calcolano correttamente la massima altezza presente a sinistra e a destra di ogni posizione. L'acqua sopra $h[i]$ è limitata dalla più bassa fra queste due barriere; sommando il contributo di tutte le posizioni si ottiene quindi la quantità totale d'acqua.

Le due tabelle e il risultato vengono calcolati mediante tre scansioni del vettore. La complessità temporale è $\Theta(n)$.

Esercizio B3 – Antipalindromo – Punti ≥ 12

Soluzione semplice

L'algoritmo genera le stringhe tramite backtracking, costruendole da sinistra a destra in un vettore $S[0 \dots n - 1]$. La procedura ricorsiva `palRec(alphabet, k, n, S, curr)` assume che il prefisso $S[0 \dots curr - 1]$ non contenga sottostringhe palindrome contigue di lunghezza maggiore o uguale a 2, e tenta di scegliere il carattere in posizione *curr*.

Per ogni carattere $c \in \text{alphabet}$, si assegna $S[curr] = c$ e si controlla se esiste un indice $start < curr$ tale che la sottostringa $S[start \dots curr]$ sia palindroma. Se ciò accade, la scelta viene scartata; altrimenti si ricorre su $curr + 1$. Nel caso $curr = 0$, il ciclo di controllo non viene eseguito, poiché una stringa di lunghezza 1 non contiene sottostringhe di lunghezza almeno 2.

La correttezza deriva dal seguente invariante: all'inizio di ogni chiamata ricorsiva, il prefisso $S[0 \dots curr - 1]$ è valido. Dopo l'assegnazione di $S[curr]$, ogni nuovo palindromo deve necessariamente terminare in *curr*, perché tutte le sottostringhe contenute interamente nel prefisso erano già state controllate. L'algoritmo verifica esattamente tutte queste nuove sottostringhe. Pertanto ogni stringa stampata è valida. Viceversa, una stringa valida non viene mai potata, e viene quindi generata.

Nel caso peggiore, l'albero di ricerca contiene $\mathcal{O}(k^n)$ nodi. In un nodo di profondità *curr*, vengono controllate $\mathcal{O}(curr)$ sottostringhe; ciascuna chiamata a `isPalindrome` costa $\mathcal{O}(curr)$. Il costo per nodo è quindi $\mathcal{O}(n^2)$ nel caso peggiore e il tempo complessivo è

$$\mathcal{O}(k^n n^2).$$

noPalindrome(**item**[] *alphabet*, **int** *k*, **int** *n*)

precondition: $n \geq 0$ and $k = |\text{alphabet}|$ and i caratteri in *alphabet* sono distinti**item**[] *S* = **new item**[$0 \dots n - 1$]`palRec`(*alphabet*, *k*, *n*, *S*, 0)

```
palRec(item[] alphabet, int k, int n, item[] S, int curr)
```

```
precondition:  $0 \leq curr \leq n$ 
if curr == n then
    print S
else
    foreach  $c \in alphabet$  do
         $S[curr] = c$ 
        boolean palindromeFree = true
        int start = 0
        while palindromeFree and start < curr do
            if isPalindrome( $S, start, curr$ ) then
                palindromeFree = false
                start = start + 1
            if palindromeFree then
                palRec(alphabet, k, n, S, curr + 1)
```

Questa soluzione, la prima che mi è venuta in mente, prende 100%.

Soluzione eccessivamente prudente

In alcuni casi non ci si è resi conto che, quando si aggiunge il carattere in posizione $curr$, tutte le sottostringhe contenute nel prefisso fino alla posizione $curr - 1$ sono già state controllate. Si ripete quindi il controllo mediante un doppio ciclo, portando il costo di ogni nodo a $\mathcal{O}(n^3)$ e il costo totale a $\mathcal{O}(n^3 k^n)$. Questa soluzione prende 90%.

```
palRec(item[] alphabet, int k, int n, item[] S, int curr)
```

```
precondition:  $0 \leq curr \leq n$ 
if curr == n then
    print S
else
    foreach  $c \in alphabet$  do
         $S[curr] = c$ 
        boolean palindromeFree = true
        for  $i = 0$  to  $curr - 1$  do
            for  $j = i + 1$  to  $curr$  do
                if isPalindrome( $S, i, j$ ) then
                    palindromeFree = false
            if palindromeFree then
                palRec(alphabet, k, n, S, curr + 1)
```

Soluzione senza pruning

In alcuni casi, sono state generate tutte le k^n possibili stringhe e il controllo di validità è stato posticipato prima della stampa. Poiché vengono generate tutte le k^n stringhe e, per ciascuna, vengono controllate $\mathcal{O}(n^2)$ sottostringhe, ognuna in tempo $\mathcal{O}(n)$, il costo totale è $\Theta(n^3 k^n)$. L'assenza di pruning rende il programma meno efficiente, prende quindi 80%.

```
palRec(item[] alphabet, int k, int n, item[] S, int curr)
```

```
precondition:  $0 \leq curr \leq n$ 
```

```
if curr == n then
```

```
    boolean palindromeFree = true
```

```
    for i = 0 to n - 2 do
```

```
        for j = i + 1 to n - 1 do
```

```
            if isPalindrome(S, i, j) then
```

```
                palindromeFree = false
```

```
    if palindromeFree then
```

```
        print S
```

```
else
```

```
    foreach c ∈ alphabet do
```

```
        S[curr] = c
```

```
        palRec(alphabet, k, n, S, curr + 1)
```

Soluzione errata

Molte soluzioni che ho visto controllano semplicemente se aggiungendo un carattere in posizione *curr*, la nuova stringa è palindroma. Questo non è sufficiente, perchè stringhe come acbc vengono generate, in quanto a, ac, acb e acbc sono tutte non palindroma, ma la sottostringa cbc è palindroma. Questo può derivare da una cattiva lettura del problema oppure da una svista. Questa soluzione, sebbene non corretta, prende 70% perché è facilmente sistemabile dopo la prima esecuzione.

```
palRec(item[] alphabet, int k, int n, item[] S, int curr)
```

```
precondition:  $0 \leq curr \leq n$ 
```

```
if curr == n then
```

```
    print S
```

```
else
```

```
    foreach c ∈ alphabet do
```

```
        S[curr] = c
```

```
        if curr == 0 or not isPalindrome(S, 0, curr) then
```

```
            palRec(alphabet, k, n, S, curr + 1)
```

Soluzione più efficiente

È sufficiente impedire la formazione di palindromi di lunghezza 2 e 3. Infatti è possibile dimostrare, per induzione, che ogni palindromo di lunghezza pari contiene un palindromo di lunghezza 2; e ogni palindromo di lunghezza dispari maggiore o uguale a 3 contiene un palindromo di lunghezza 3. È quindi sufficiente controllare soltanto:

- il suffisso di lunghezza 2, verificando se $S[curr] = S[curr - 1]$;
- il suffisso di lunghezza 3, verificando se $S[curr] = S[curr - 2]$.

Se entrambi i controlli falliscono, il nuovo prefisso è ancora valido.

```
noPalindrome(item[] alphabet, int k, int n)
```

```
precondition:  $n \geq 0$  and  $k = |alphabet|$  and i caratteri in alphabet sono distinti
```

```
item[] S = new item[0 .. n - 1]
```

```
palRec(alphabet, k, n, S, 0)
```

```
palRec(item[] alphabet, int k, int n, item[] S, int curr)
```

```
precondition:  $0 \leq curr \leq n$ 
```

```
if curr == n then
```

```
    print S
```

```
else
```

```
    foreach  $c \in alphabet$  do
```

```
         $S[curr] = c$ 
```

```
        boolean palindromeFree = ( $curr < 1$  or  $S[curr] \neq S[curr - 1]$ )
```

```
        palindromeFree = palindromeFree and ( $curr < 2$  or  $S[curr] \neq S[curr - 2]$ )
```

```
        if palindromeFree then
```

```
            palRec(alphabet, k, n, S, curr + 1)
```

Ogni tentativo di estensione richiede ora tempo $\mathcal{O}(1)$. Un limite superiore immediato per il tempo di esplorazione è quindi $\mathcal{O}(k^n)$. Tenendo conto del costo di stampa, il tempo è $\mathcal{O}(n)$ per ogni stringa prodotta e quindi il costo è $\mathcal{O}(nk^n)$.

Questa soluzione, che riduce il costo di un fattore n , prende 110%.